

Optimus-Cost-Agent

Enterprise Architectural Blueprint

Adaptive Agent Reasoning Swarms & Evolutionary Governance Contracts

Version 2.15

Architected by: Vibhanshu Agarwal

1. Executive Summary

Conceptual framing adapted from evolutionary software boundaries formulated by Neal Ford, Rebecca Parsons, and Patrick Kua (O'Reilly, 2026). The Optimus-Cost-Agent v2.15 transitions from a basic token-routing utility to a formal Harness Engineering framework designed to govern non-deterministic generation via deterministic software boundaries. Standard AI agents are fundamentally bounded by the Dreyfus model's skill acquisition ceiling; they cannot brute-force their way past the competent or proficient barrier into true architectural consistency through raw scaling alone. Optimus raises the agent's effective architectural competence through deterministic constraints and feedback by wrapping logical model abstraction inside an opinionated local-first Python ACP server governed by automated, multi-tier architectural fitness functions and explicit request-level cost attribution.

2. Harness Engineering & Context Optimization

To protect the token window from context bloat while systematically mitigating hallucination, the agent structure enforces a strict division between Feedforward Controls and Feedback Sensors:

- **Feedforward Controls (Pre-Generation):** Before a prompt payload hits the downstream model, the Context Optimization Node constructs structural boundaries. It strips code down to AST-based functional slices, structures static filesystem metadata layouts at the head of the prompt window to maximise GLM-5.2 input caching, and injects succinct, unambiguous constraints using an Architecture Definition Language (ADL) dialect to minimise context attention allocation.
- **Feedback Sensors (Post-Generation Evaluation):** Once the primary model provider emits a candidate patch, the engine catches the text stream inside a validation loop. It intercepts execution blocks and evaluates them via automated, triggered fitness functions before any source mutations are applied to the active working tree.

3. Automated Architectural Fitness Functions

Optimus treats system governance not as a passive logging layer, but as an active, objective integrity check on core architectural traits. Phase 1 implements highly targeted, local validation primitives organised across five operational planes: Scope (Atomic), Automation (Automated), Invocation (Triggered), Return (Static), and Activation (Proactive). These gates intercept code generations mid-flight to fix structural deviations before changes lock into local Git trees.

4. Data Governance Plane & Local Storage Boundaries

To support enterprise compliance review, data within the local container is sharply bounded. Unparsed source code is strictly prohibited from entering persistent vector indexes. The storage engine uses standard Redis HASH structures to catalogue structural class signatures, summaries, and relative file paths. Numeric performance histories are channelled separately to a Redis TimeSeries cluster (tracking tokens_input, tokens_output, cost_usd) tagged by unique run_id strings and protected by a hard 30-day retention window policy.

5. Request-Level Cost Attribution

Attribution calculations bypass tokeniser estimators, treating the provider usage response as the primary request-level cost record, supported by versioned pricing snapshots. All gateway usage fields — gateway_request_id, provider, cache_hit, billing_units, and cost_usd — are parsed directly from the gateway response envelope, never estimated post-hoc.

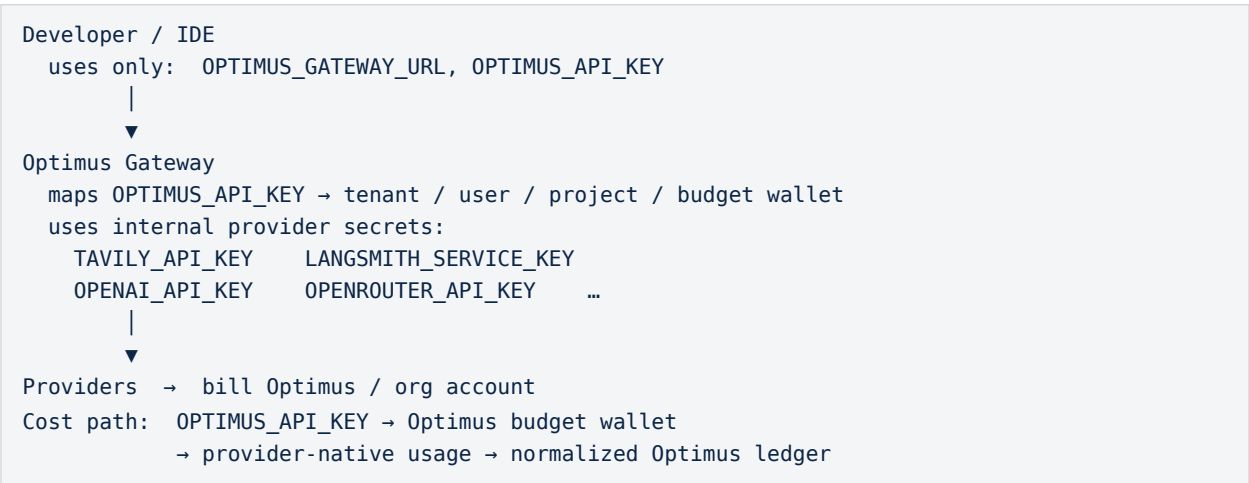
Execution Metric	Optimus Cascaded Route	Claude Sonnet 4.6 (Direct)	Claude Opus 4.8 (Direct)
Calculated Task Cost	\$0.018 (Haiku Triage + GLM Cache Hit)	\$0.045	\$0.150
Source of Truth	API usage response object	Counterfactual Reroute Simulation	Counterfactual Reroute Simulation
Cache Accounting	Active Hit Registered (\$0.26/M)	N/A (Full Input Evaluation)	N/A (Full Input Evaluation)

5A. Provider-Cost Normalization & Single-Key Wallet Model

The Optimus API key is the only developer-facing credential. It maps to an internal tenant/user/project budget wallet. Upstream provider keys for LLMs, Tavily, LangSmith, and any other vendor are owned by the Optimus Gateway and are never configured locally. Third-party vendors do not natively consume Optimus credits — each expects its own vendor credential — so the friction-free model is that developers never see those keys: OPTIMUS_API_KEY authenticates to the Optimus Gateway, and the Gateway owns all downstream provider keys, usage mapping, billing normalization, and budget enforcement.

From a cost perspective, OPTIMUS_API_KEY is not a physical upstream key; it is an internal account / wallet key. Optimus credits are an internal abstraction over real, heterogeneous vendor costs (LLM tokens, Tavily credits, and trace / observability allocation). They do not remove the need for upstream vendor accounts, but they remove the need for every developer to configure and fund them. The Gateway converts each provider's native usage into one normalized Optimus ledger.

Normalized-Cost Mapping



This preserves the thesis: one key, one budget, one ledger, with many providers behind the curtain. Tavily is treated as a first-class Gateway tool and LangSmith as a Gateway-managed observability sink;

neither becomes a local developer dependency. Executable mechanics — endpoint shapes and the normalized ledger schema — are specified in the LLD (§0A, §10A).

6. Deterministic Data-Flow Architecture (Phase 1 MVP)

The updated system design integrates closed-loop Harness Engineering directly into the orchestration sequence:

- [1] User Request Initiated (IntelliJ IDEA Client).
- [2] Ingress over ACP Protocol over StdIn JSON-RPC to the Optimus ACP Server Daemon (Asyncio Pipeline).
- [3] Read Topology & Structural Memory Maps from Local Redis State Store (HASH Schema Core Pull), which returns active code signatures and cached map assets.
- [4] Processing inside the Context Optimization Node [FEEDFORWARD CONTROLS] executing AST Slicing, Polyglot Fallbacks, ADL Constraint Prompt Injection, and Prompt Anchoring for Cache Tuning.
- [5] Emits minimal context prompt payload to the LangGraph Stateful Router (Claude Haiku 4.5 Triage Gate) for escalation route evaluation.
- [6] Delivery to Optimus AI Gateway → Frontier Model Providers (GLM-5.2 Primary Generation Loop), returning the raw unified diff patch alongside the API usage object.
- [7] Validation inside the Automated Fitness Function Engine [FEEDBACK SENSORS] conducting atomic structural checks via PyTestArch and static code metrics gate assertions (MI / Complexity Check). On FAIL: loops back to Step [6]; on PASS: approves code patch execution path.
- [8] Storage inside the Local Storage Engine Persistence Layer via Async TS.ADD Numeric Telemetry Tracking (RedisTimeSeries) and Async HSET Workspace Metrics Serialisation (Redis HASH Architecture Indexes).
- [9] Real-Time FinOps Console Panel display for IDE Cost Dashboard Live Update Screen.

7. Agent Operating Modes & Trust Framework

To guarantee user trust and operational predictability, the control plane implements strict execution state isolation:

- **Plan/Chat Mode:** Advisory-only mode. The agent may inspect context, discuss requirements, propose plans, and produce diffs or snippets for review, but it must not mutate the repository, filesystem, external services, or user/project state. Internal cost, audit, and performance telemetry is append-only and explicitly allowed.
- **Agent Mode:** Execution mode. The agent is write-authorized and empowered to actively modify code, run tools, create files, update tests, and apply patches within approved workspace boundaries. Modifications apply to the working tree only after clearing all composite fitness engine gates.
- **Code Generation Scope Classification:** Delineates advisory inline fragments from implementation deliverables based on clear architectural criteria:
 - **INLINE_SNIPPET:** Explanatory text fragments under 15 lines that touch zero core packages and create/delete no files.

- **PATCH_PROPOSAL:** Token streams over 15 lines that edit existing code assets exclusively without file structural modifications.
- **FILE_MUTATION:** Single-file generation operations, re-writes, or file deletions that require shell checking or tool alignment.
- **MULTI_FILE_CHANGESET:** High-risk operations touching shared/core architecture modules, crossing distinct directory roots, or altering multi-package configuration environments.

8. Tool Governance & Evidence Acquisition

Optimus treats tool use as policy-driven first, model-requested second: the model may ask for evidence, but the harness alone decides whether a tool is allowed, necessary, and cost-justified. This keeps tool invocation aligned with the same cost-control discipline applied to model routing.

Tool Stack (Phase 1)

Tool Class	Examples	Invocation Style
Local repo discovery	rg, file reader, AST parser, dependency graph, git diff	Deterministic, cheap, first-line
Validation gates	Tests, lint, type check, architecture fitness checks, security scan subset	Deterministic after patch generation
Patch workspace	Shadow apply, diff parser, rollback, working-tree apply	Agent mode only
Cost telemetry	RedisTimeSeries, run metadata hash, pricing snapshot	Always-on internal telemetry
Web evidence	Gateway search + extract (Tavily-backed)	Policy-triggered, read-only
Package/version metadata	PyPI, npm, Maven Central, GitHub releases, OSV.dev	Only when dependency/version/security task requires it
External execution	Shell, build, test, package install	Agent mode only, gated

Deterministic Invocation Policy

A ToolInvocationPolicy matrix maps task signals to permitted tool classes, so "search the web" is never a casual model impulse. Representative triggers:

- User explicitly asks for current/latest/external facts → web search.
- Task mentions library/API/framework behaviour not present in the repo → official-doc web search.
- Task touches dependencies → package registry lookup.
- Task touches security/CVEs → OSV or advisory lookup.
- Task asks for a local code change → local repo search first, no web by default.
- Model makes an architectural claim without inspected evidence → file inspection or Assumption Ledger entry.
- Patch generated → shadow apply + fitness gates.

- Fitness gate fails → reflection or targeted local inspection.
- Plan/Chat mode active → read/search tools only, no mutation tools.

Each web-evidence request carries an explicit reason code (e.g. API_DOCS_OUTDATED, CURRENT_FACT, SECURITY_ADVISORY, USER_REQUESTED, PACKAGE_VERSION) plus an allowed-domain list, result cap, and search-depth setting, so external calls remain auditable and cost-bounded rather than ad hoc.

Web evidence acquisition is wrapped behind Optimus-owned typed tools rather than exposing a third-party search API directly to the model. This lets the harness enforce allowed domains, timeouts, retries, result caps, and cost telemetry uniformly. Key rule: local evidence first, external evidence only when policy-triggered, mutation only after mode and fitness gates pass.

9. Adaptive Agent Execution Strategy & Rigor Policy

Optimus applies adaptive execution strategies to balance cost, rigor, and reliability. The agent selects Direct Execution, Plan-and-Execute, ReAct, or Reflection based on task ambiguity, mutation risk, and validation outcomes. Reflection and deep planning are not default behaviours; they are exception-triggered circuit breakers invoked only when risk signals, failed validation, or insufficient evidence require additional rigor.

Low-Cost Hallucination Controls

- **Evidence-Bound Decisions:** No architectural claim may be promoted to an implementation decision unless backed by inspected context or recorded as an explicit entry in a machine-readable Assumption Ledger.
- **Rigor Budget Matrix:** Bounded request tiers (LOW, MEDIUM, HIGH) prevent the agent from spending tokens on ceremonial planning loops for simple edits:
 - **LOW:** Cap at 5 tool calls, 0 reflection passes, max 100k context tokens, preferred HAIKU model tier.
 - **MEDIUM:** Cap at 10 tool calls, 1 reflection pass, max 150k context tokens, preferred PRO model tier.
 - **HIGH:** Cap at 15 tool calls, 3 reflection passes, max 250k context tokens, preferred PRO model tier.
- **Trigger-Based Reflection:** Reflection initialises only when validation loops trip a failure signature or code touches shared/core architecture modules, preventing quiet token-doubling on every nominal request.
- **Assumption Ledger:** Bypasses token-heavy text summaries by generating an explicit JSON ledger identifying unverified assumptions, confidence thresholds, and specific validation files needed to clear uncertainty.
- **The Escalation Ladder:** Bounds agent tool invocation paths to escalate strictly through a cost-minimising chain: Provided Context → File Inspection → Plan → Execute → Validate → Targeted Reflection → User Escalation.

10. Architectural Control Flow

This section provides visual reference diagrams for the three primary control flow concerns in Optimus: where the system boundaries sit, how the governance loop operates end-to-end, and

how the architecture evolves across phases.

A. System Context Diagram

The system context places Optimus within its operating environment. All provider calls flow through the Optimus AI Gateway; the local agent holds only a single Optimus credential. The IDE plugin and local agent communicate over ACP (JSON-RPC over stdin/stdout).

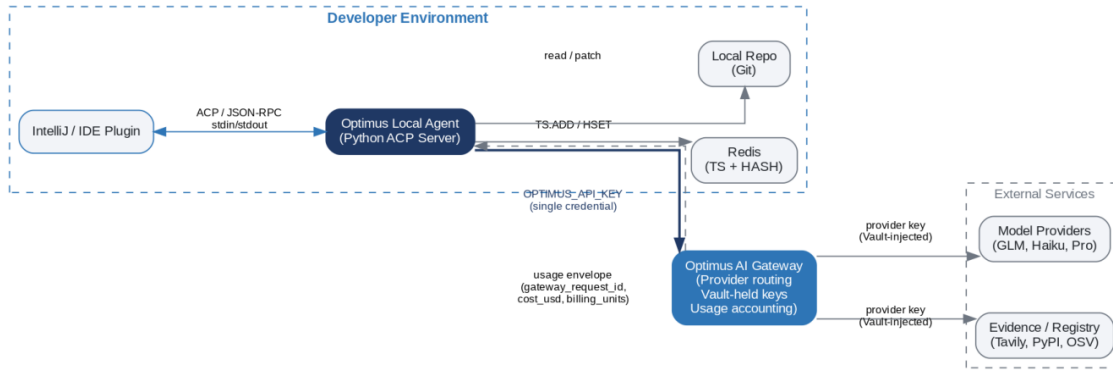


Figure 10.A · System Context — single Optimus credential; all provider keys Vault-held server-side.

B. Governance Loop Diagram

Every user request traverses the full governance loop. The loop enforces that no model output reaches the working tree without passing through both the feedforward constraint layer and the feedback fitness gate layer.

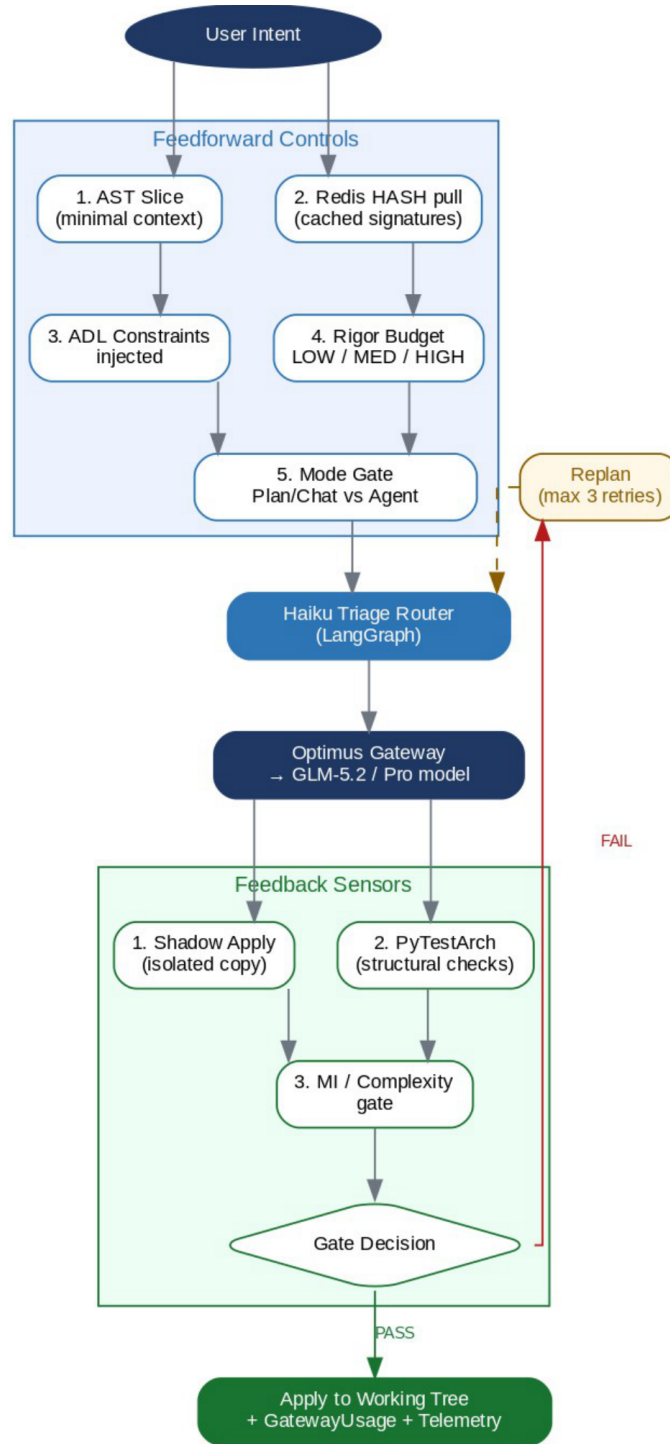
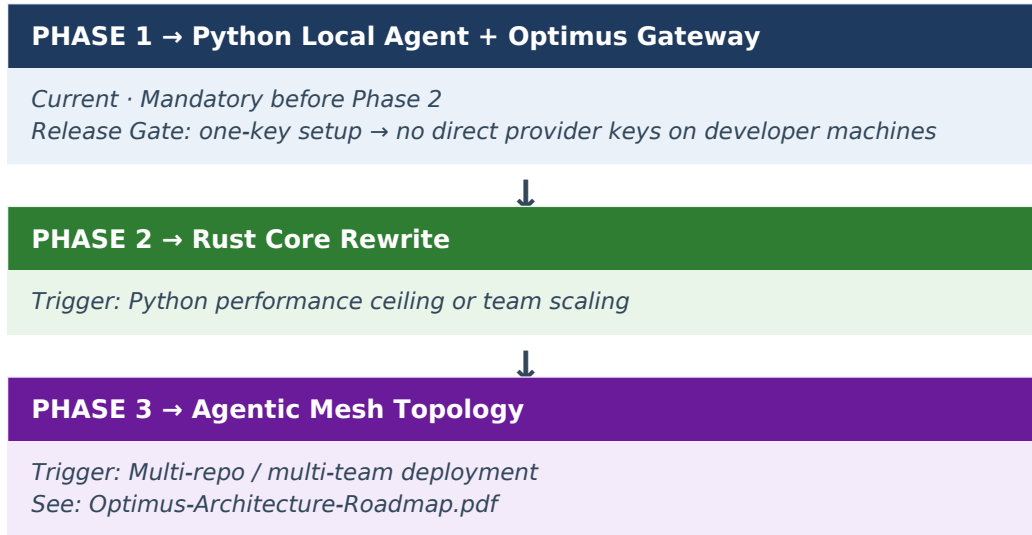


Figure 10.B · Governance Loop — feedforward controls, triage routing, feedback sensors, mutation gate.

C. Phase Evolution Diagram

Optimus is designed as a three-phase architecture. Phase 1 is mandatory before any later phase.



D. Where Cost Control Happens

Cost control is not a single checkpoint — it is enforced at four distinct points in the request lifecycle, each with a different mechanism.

Control Point	Mechanism	What It Prevents
1. Pre-prompt (Feedforward)	AST slicing + cache anchoring + ADL constraints	Unnecessary tokens sent to expensive models.
2. Router (Triage Gate)	Haiku classifies task → routes to cheapest sufficient model	Over-routing simple tasks to Pro/Opus tier.
3. Rigor Budget (Runtime)	LOW/MEDIUM/HIGH tier caps tool calls + reflection passes	Token-doubling on ceremonial planning loops.
4. Post-call (Attribution)	Gateway usage object → EvidenceLedger → RedisTimeSeries	Silent cost accumulation; enables per-run audit.

E. Where Hallucination Control Happens

Hallucination is controlled through a defence-in-depth stack. No single gate is sufficient; all five layers must be active simultaneously.

Layer	Mechanism	What It Catches
1. Context Constraint	ADL rules injected pre-prompt; AST slices only relevant code	Irrelevant context that confuses the model.
2. Evidence Gate	Assumption Ledger: every architectural claim must be inspected or logged	Unverified claims promoted to implementation decisions.
3. Tool Policy	ToolInvocationPolicy blocks casual web calls; reason codes required	Model hallucinating external facts without evidence.

Layer	Mechanism	What It Catches
4. Fitness Functions	PyTestArch + MI gates reject structurally invalid patches	Generated code that violates architectural contracts.
5. Reflection Circuit Breaker	Reflection only on gate failure or MULTI_FILE_CHANGESET	Silent acceptance of bad patches without re-inspection.

11. Optimus AI Gateway — Phase 1 Mandatory

The Optimus AI Gateway is Phase 1 mandatory infrastructure. All traffic from the local agent — model completions and every tool call — flows through the gateway from day one. There is no supported configuration in which the local agent calls a provider directly.

The gateway holds all provider credentials server-side in a Vault. The local agent holds only two values: `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`. This eliminates credential sprawl on developer machines and centralises cost attribution.

Gateway Responsibilities

- Route model completion requests to the correct provider (GLM, Claude Haiku/Sonnet/Opus) based on the model alias in the request.
- Inject provider API keys server-side from Vault; no key is ever transmitted to the local agent.
- Record `gateway_request_id`, `provider`, `cache_hit`, `billing_units`, and `cost_usd` in every response envelope.
- Enforce domain allowlists for web search and extract requests, independent of local agent policy.
- Revalidate budgets, call caps, and tool policies server-side as a defence-in-depth layer (Section 9D of LLD).
- Enforce origin allowlist: local agent `gateway_url` must resolve to a trusted origin; rogue gateway attacks are blocked via `production_mode` + signed tenant profile.

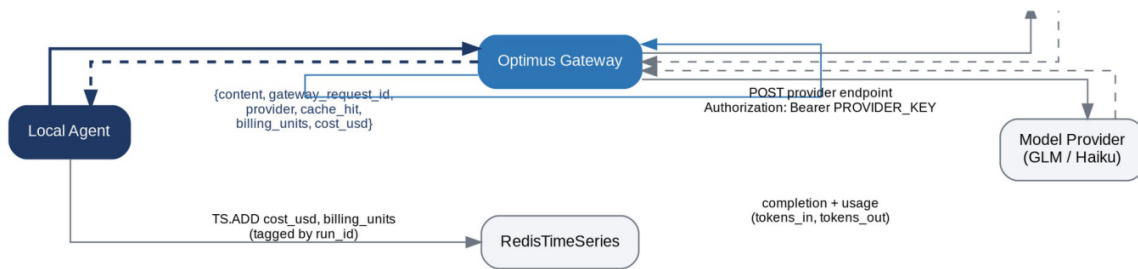
Single-Credential Model

```
# Local agent environment – the only credentials required
OPTIMUS_GATEWAY_URL=https://gateway.optimus.ai
OPTIMUS_API_KEY=opt_live_xxx

# Gateway holds (server-side, Vault) – never on local machine
OPENAI_API_KEY=sk-...
GLM_API_KEY=...
TAVILY_API_KEY=tvly-...
LANGSMITH_SERVICE_KEY=ls-...      # observability, server-side only
OPENROUTER_API_KEY=or-...
```

Gateway Request / Response Sequence

Every model completion and tool call follows the same sequence: the local agent sends a single authenticated request to the gateway, which injects provider credentials server-side, forwards to the upstream provider, and returns a unified response envelope containing the model output plus GatewayUsage accounting fields.

Figure 11.1 · Gateway Request / Response Sequence

Swimlane: Local Agent · Optimus Gateway · Vault · Model Provider · RedisTimeSeries.

The Local Agent opens one connection to the Optimus AI Gateway and sends a completion request; the Gateway injects the correct provider API key from Vault, routes to the selected model (GLM, Haiku, Sonnet, or Opus), and returns a GatewayUsage envelope with `gateway_request_id`, `provider`, `cache_hit`, `billing_units`, and `cost_usd`. The Local Agent writes this object to RedisTimeSeries for 30-day cost attribution. No provider credentials ever leave the Gateway server.

11A. Trace Observability

Phase 1 uses LangSmith for trace observability across planning, gateway calls, tool invocation, validation, retries, and final response generation. At the architectural level, LangSmith is the trace sink — but its credentials are managed by the Gateway / deployment layer to preserve one-key setup. LangSmith is an observability and production-debugging tool, not a test-coverage tool, and does not contribute to the coverage release gate.

LangSmith cost is recorded the same way as any other provider cost, but the commercial model varies. Depending on the plan, the Gateway records either provider-native trace usage (e.g. trace / span / event counts) or an allocated / amortized observability cost where billing is seat- or subscription-based. In all cases the figure is normalized into the same Optimus ledger as token and tool costs (see §5A; LLD §10A for field-level detail).

One-key principle. LangSmith — and every other vendor — credential lives in Gateway / deployment secrets, never in local developer config. Local dev requires only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`.

12. Testing & Quality Gates

Phase 1 targets a **minimum 80% Python production-code coverage** gate, with materially higher expectations for safety-critical modules (mode enforcement, gateway usage accounting, evidence reconciliation, retry/fail-closed logic, policy enforcement). Coverage is a release gate, not a quality substitute.

Agent-quality and AI-safety evaluations — DeepEval (task-completion, hallucination, faithfulness, role-adherence), Ragas (retrieval / evidence-quality), and PyRIT (adversarial / red-team) — together with LangSmith trace observability are tracked separately and do not count toward the coverage metric. This section states architectural policy only and may summarize the target; the Test Strategy document (§8A) remains the authoritative source of truth for the coverage threshold, the measurement-tool taxonomy, and trace observability.

Concern	Tooling	Role in gates
Code coverage	coverage.py, pytest-cov	Release gate — min 80% Python production code
Agent quality eval	DeepEval, Ragas	Tracked separately; not counted toward coverage
Adversarial / red-team	PyRIT	Tracked separately; not counted toward coverage
Trace observability	LangSmith (Gateway-managed key)	Debugging / regression analysis; not a coverage metric

13. Agent Execution Safety & Guardrails

Agent execution safety is enforced through a layered guardrail model that wraps every tool call between the moment the agent decides to act and the moment the action takes effect: **mode constraints** (§7), a **permission policy** (allow/deny lists with a human-approval path), a **pre-tool guard** that deterministically validates shell, file, MCP, and network calls, **sandboxed execution**, **post-tool audit** against the Gateway cost and trace ledgers (§5A, §11, §11A), and **commit/CI gates** (§12). The same plane governs two execution patterns — **bounded goal-driven agent loops** and **curated workflow skills** — so that long-running or procedure-driven work inherits the identical constraints and the same Gateway budget policy.

The premise is not that the agent is malicious but that it is a reward-hacky problem solver: under pressure to reach "done" it will reach for `chmod 777`, `curl ... | sh`, force-push to main, or commenting out a failing assertion. Guardrails remove those shortcuts from its reach. Most of the model is deterministic and zero-token; a cheap, budgeted classifier routed through the Optimus Gateway is consulted only for borderline cases, never on the hot path of an already-allowed or already-denied call.

The detailed policy model, component contracts, and control-flow diagram are maintained in the companion specification **Agent Execution Guardrails & Workflow Strategy v1.0** (control-plane diagram in its §1; LLD component contracts in its §10; test categories in its §11). The corresponding implementation contracts are specified in **LLD §12** and the test categories in **Test Strategy §14**. This section is a cross-cutting reference only.